ZeroPoint

Edge AI Gateway & Cost Governance Proxy

Software Requirements Specification (SRS)

Product	ZeroPoint
Version	1.0
Status	Technical Baseline
Standards	IEEE 830 / ISO/IEC 29148
Primary Runtime	Cloudflare Workers / V8 Edge Isolates
Implementation	TypeScript

1. Introduction

1.1 Purpose

ZeroPoint is an Edge AI Gateway and Cost Governance Proxy designed to provide real-time financial controls, usage governance, and runaway-agent protection for AI-powered SaaS applications. Modern AI SaaS products commonly expose LLM functionality through subscription plans while incurring variable provider costs. High-volume users, autonomous agents, retries, and incorrectly configured loops can therefore generate API costs that materially exceed subscription revenue.

Traditional dashboards provide primarily post-hoc observability. ZeroPoint instead operates directly in the request path and provides pre-flight enforcement and real-time circuit breaking.

• Authenticate incoming proxy requests.

• Associate requests with individual end users.

• Evaluate spending limits before an LLM request is forwarded.

• Forward authorized requests to supported LLM providers.

• Process streaming responses without material streaming latency.

• Track token usage and financial consumption asynchronously.

• Detect abnormal spending velocity and repeated request patterns.

• Automatically open a circuit breaker when thresholds are exceeded.

• Provide administrative spending and circuit-breaker analytics.

• Minimize stored user content through a Zero-Trace Privacy Policy.

1.2 Scope

1.2.1 In Scope

• Proxy authentication

• End-user identification

• Tenant isolation

• Provider credential resolution

• Pre-flight spending enforcement

• Request forwarding

• Streaming response forwarding

• SSE stream inspection

• Token/usage extraction

• Asynchronous usage settlement

• Redis-based real-time spend tracking

• Sliding-window burn-rate analysis

• Request-pattern hashing

• Circuit-breaker enforcement

• Developer webhook notifications

• Persistent usage metadata

• Administrative analytics

• API-key provisioning

• Audit and request metadata

1.2.2 Out of Scope

• User registration and authentication

• Subscription billing and payment processing

• Customer identity management

• Application business logic

• Prompt construction

• Agent orchestration

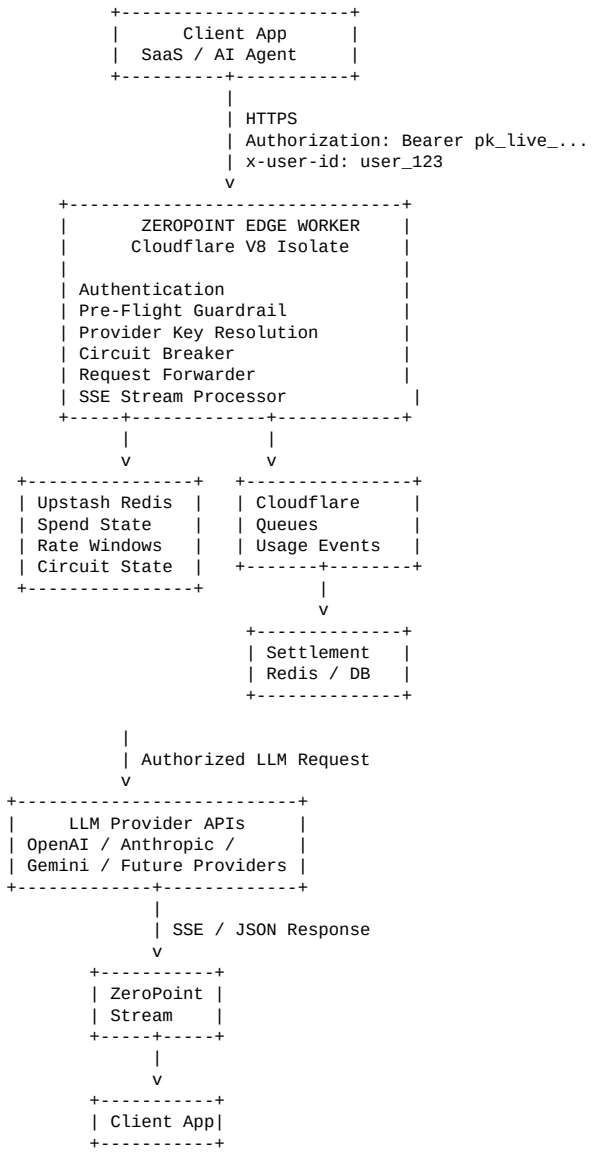
• Model-selection UI

• Provider-specific model behavior

• Long-term prompt/output storage

ZeroPoint is a governance and proxy layer, not an application identity provider or billing platform.

1.3 System Architecture



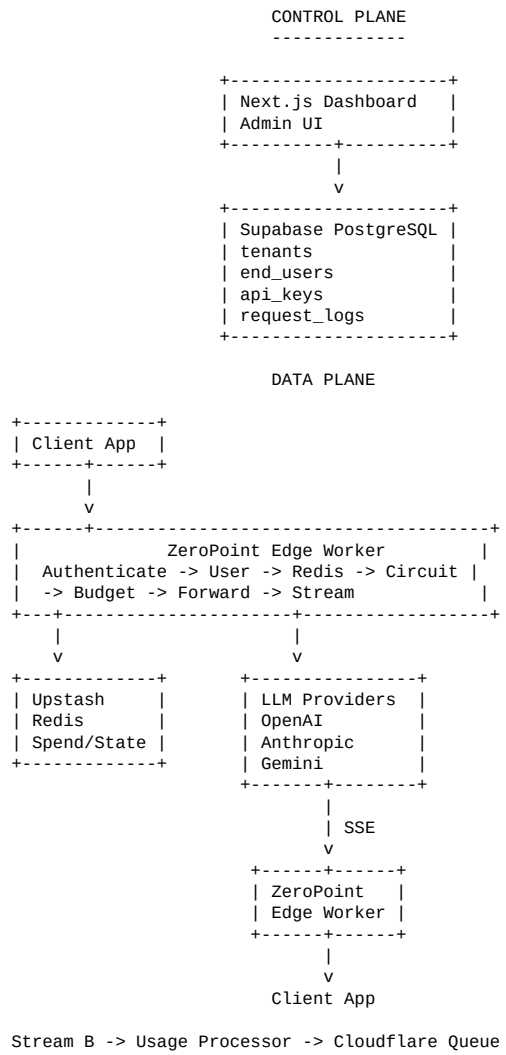
1.4 Definitions, Acronyms & Abbreviations

Term	Definition
AI	Artificial Intelligence
API	Application Programming Interface

Term	Definition
AES	Advanced Encryption Standard
GCM	Galois/Counter Mode
SSE	Server-Sent Events
LLM	Large Language Model
SaaS	Software as a Service
TTL	Time To Live
SHA-256	Secure Hash Algorithm, 256-bit
V8 Isolate	Lightweight JavaScript execution environment used by Cloudflare Workers
Pre-Flight Guardrail	Budget and policy validation performed before an LLM request reaches a provider
Dual-Stream Piping	Duplication of a readable response stream using ReadableStream.tee() so one branch is delivered to the client while another is inspected
Velocity Circuit Breaker	Protective mechanism that blocks a user when AI expenditure exceeds a configured spending velocity threshold
Tenant	SaaS organization using ZeroPoint
End User	Individual application user governed by ZeroPoint

2. Overall Description

2.1 Product Perspective & Data Flow



```
          -> Redis atomic settlement + PostgreSQL logs
```

2.2 User Classes & Characteristics

2.2.1 SaaS Developers / Tenants

SaaS organizations configure provider credentials, proxy keys, budgets, thresholds, analytics, and alerting. Expected technical proficiency is intermediate to advanced.

2.2.2 End Users

End users interact with the client SaaS application and are identified to ZeroPoint through the x-user-id request header.

2.3 Operating Environment & Constraints

• Cloudflare Workers / V8 Isolates.

• TypeScript and Web Platform APIs.

• Target maximum working memory: approximately 128 MB.

• Target ZeroPoint pre-flight overhead: under 10 ms.

• Target streaming overhead: under 10 ms.

• No conventional Node.js runtime or Node.js built-ins.

• No unrestricted filesystem access.

• Avoid full response buffering and large in-memory structures.

3. Specific Functional Requirements

Phase 1 — Core Edge Proxy & Pre-Flight Budget Guardrails

FR-1.1 Header Authentication

The Edge Worker shall authenticate requests using Authorization: Bearer <PROXY_KEY> and x-user-id: <USER_ID>. Missing or invalid credentials shall return HTTP 401 Unauthorized.

```
Incoming Request
      |
      v
Read Authorization
      |
      +---- Missing ----> 401
      |
      v
Extract Bearer Token
      |
      v
Hash Proxy Key
      |
      v
Resolve Tenant / Key
      |
      +---- Invalid ----> 401
      |
      v
Read x-user-id
      |
      +---- Missing ----> 401
      |
      v
Continue
```

FR-1.2 Pre-Flight Budget Check

Before forwarding an LLM request, ZeroPoint shall compare current user spend against the configured budget. If current_spend >= budget_limit, ZeroPoint shall terminate processing before contacting the LLM provider and return HTTP 429 Too Many Requests.

```
Request -> Authenticate -> Load Current Spend
                        -> current_spend >= budget?
                           YES -> 429
                           NO  -> Continue
```

FR-1.3 Header Transformation

After successful authentication and policy checks, ZeroPoint shall replace the client proxy credential with the appropriate provider credential. The proxy key shall never be forwarded to an external LLM provider.

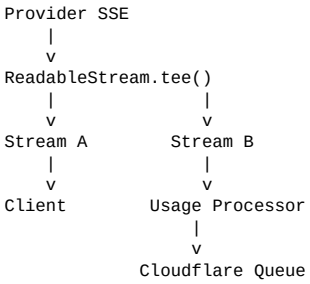
FR-1.4 Provider Routing

The proxy shall support provider-specific routing for OpenAI, Anthropic, Gemini, and future providers through modular provider adapters.

Phase 2 — Zero-Lag Dual-Stream Parsing & Asynchronous Ingestion

FR-2.1 Dual-Stream Piping

When an LLM provider returns a streaming response, ZeroPoint shall avoid full buffering and duplicate the response using ReadableStream.tee(). Stream A shall be delivered to the client; Stream B shall be inspected for usage metadata.

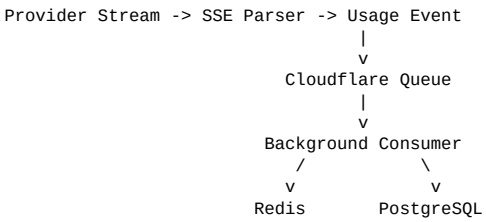


FR-2.2 SSE Chunk Decoder

The analytics branch shall decode chunks incrementally using TextDecoder and correctly handle partial SSE frames, multiple events per chunk, events split across chunks, provider-specific formats, and terminal [DONE] events where applicable.

FR-2.3 Out-of-Band Queue Settlement

Usage events shall be dispatched to Cloudflare Queues. Background consumers shall atomically increment Redis spend and persist durable request metadata.



FR-2.4 Idempotent Settlement

Each usage event shall contain a unique request_id. Queue retries shall not result in duplicate financial accounting.

Phase 3 — Runaway Agent Loop Circuit Breaker ("WOW" Feature)

FR-3.1 Burn-Rate Tracking

ZeroPoint shall maintain a 60-second sliding spending window using Redis sorted sets. Entries shall contain timestamps and costs. Expired entries shall be removed before calculating current velocity.

```
velocity:{tenant_id}:{user_id}

ZADD velocity:<key> <timestamp> <cost>
ZREMRANGEBYSCORE velocity:<key> <oldest> <now-60s>
```

FR-3.2 Velocity Hard Stop

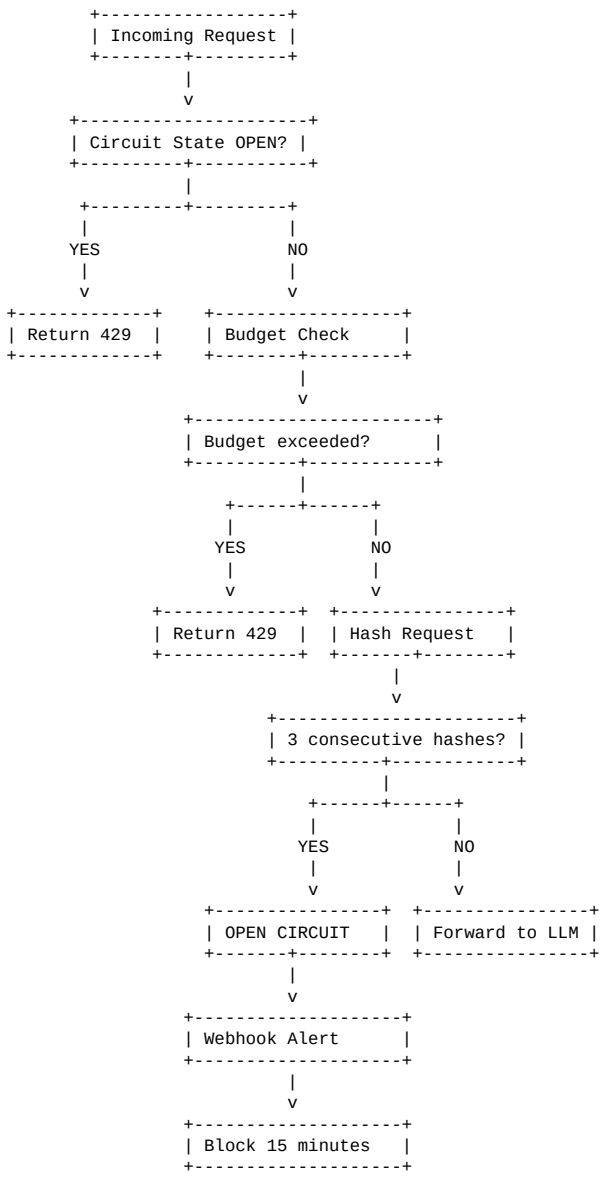
The default circuit-breaker threshold shall be $2.00 per 60 seconds. If the calculated spend rate exceeds this threshold, the circuit shall open.

FR-3.3 Pattern Hash Matching

ZeroPoint shall generate a SHA-256 hash of a normalized request representation and retain the last five hashes per user. Three consecutive matching hashes shall trigger the loop-detection circuit breaker.

FR-3.4 State Machine Actions

When triggered, ZeroPoint shall set the circuit state to OPEN, block requests for 15 minutes, record the event, and dispatch a developer webhook.



FR-3.5 Circuit States

The initial implementation shall use CLOSED and OPEN states with a 15-minute expiration. A future HALF_OPEN state may support controlled recovery probes.

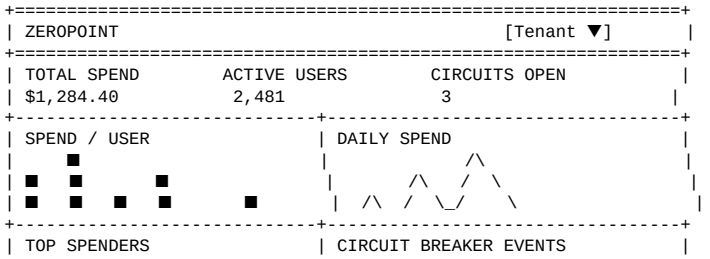
Phase 4 — Administrative Analytics Dashboard

FR-4.1 Key Provisioning

Administrators shall be able to generate proxy keys in pk_live_<random_secret> format. The complete key shall be shown once and only its SHA-256 digest shall be stored.

FR-4.2 Real-Time Visualizations

The dashboard shall expose spend-per-user charts, top spenders, request/token metrics, and active circuit-breaker events.



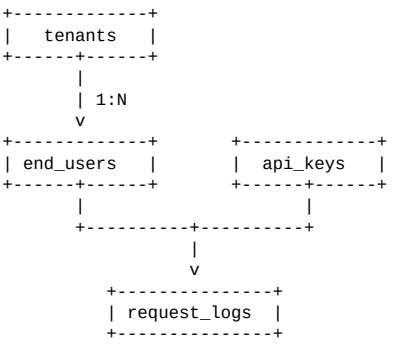
```
| u_42       $18.20           | u_123  VELOCITY       OPEN       |
| u_19       $14.70           | u_901  LOOP           OPEN       |
+-----------------------------+----------------------------------+
```

4. Data Schemas & Database Design

4.1 Database Principles

PostgreSQL shall be the durable system of record for tenants, end users, API-key metadata, and request metadata. Redis shall hold low-latency counters, sliding windows, circuit state, and rate-limit state.

4.2 Entity Relationship Model



4.3 Full SQL Schema

```
CREATE TABLE tenants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,
    default_budget_usd NUMERIC(12,4) NOT NULL DEFAULT 20.0000,
    velocity_limit_usd NUMERIC(12,4) NOT NULL DEFAULT 2.0000,
    velocity_window_seconds INTEGER NOT NULL DEFAULT 60,
    circuit_breaker_duration_seconds INTEGER NOT NULL DEFAULT 900,
    webhook_url TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE end_users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    external_user_id TEXT NOT NULL,
    budget_limit_usd NUMERIC(12,4),
    current_spend_usd NUMERIC(12,4) NOT NULL DEFAULT 0,
    status TEXT NOT NULL DEFAULT 'active'
        CHECK (status IN ('active','blocked','suspended')),
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (tenant_id, external_user_id)
);

CREATE TABLE api_keys (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    key_hash CHAR(64) NOT NULL,
    key_prefix TEXT NOT NULL,
    name TEXT,
    status TEXT NOT NULL DEFAULT 'active'
        CHECK (status IN ('active','revoked')),
    last_used_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    revoked_at TIMESTAMPTZ,
    UNIQUE (key_hash)
);

CREATE TABLE request_logs (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id UUID NOT NULL UNIQUE,
    tenant_id UUID NOT NULL REFERENCES tenants(id) ON DELETE CASCADE,
    end_user_id UUID REFERENCES end_users(id) ON DELETE SET NULL,
    provider TEXT NOT NULL,
    model TEXT,
    status_code INTEGER,
    input_tokens BIGINT NOT NULL DEFAULT 0,
    output_tokens BIGINT NOT NULL DEFAULT 0,
    total_tokens BIGINT NOT NULL DEFAULT 0,
    estimated_cost_usd NUMERIC(14,8) NOT NULL DEFAULT 0,
    usage_source TEXT CHECK (
        usage_source IN ('provider_reported','estimated','unknown')
    ),
```

```
    latency_ms INTEGER,
    streaming BOOLEAN NOT NULL DEFAULT FALSE,
    circuit_breaker_triggered BOOLEAN NOT NULL DEFAULT FALSE,
    circuit_breaker_reason TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_end_users_tenant ON end_users(tenant_id);
CREATE INDEX idx_request_logs_tenant_created
ON request_logs(tenant_id, created_at DESC);
CREATE INDEX idx_request_logs_user_created
ON request_logs(end_user_id, created_at DESC);
CREATE INDEX idx_request_logs_provider ON request_logs(provider);
CREATE INDEX idx_api_keys_tenant ON api_keys(tenant_id);
```

5. Security, Privacy & Non-Functional Requirements

NFR-5.1 Latency Overhead

Pre-flight checks shall target less than 10 ms of ZeroPoint application overhead. Streaming processing shall target less than 10 ms. These targets exclude client/provider network latency and LLM generation time.

NFR-5.2 Security & Data Privacy

• AES-256-GCM encryption for sensitive provider credentials at rest.

• SHA-256 hashing for proxy keys.

• No prompt or generated-output persistence.

• No provider credentials exposed to clients.

• No credentials in application logs.

• Tenant-scoped data and Redis keys.

• TLS for external communications.

• Least-privilege administrative access.

NFR-5.3 Reliability & Fail-Open Strategy

The default Redis failure strategy shall be Fail-Open to prevent a temporary governance-infrastructure outage from crashing a customer production application. Fail-open events shall be observable through operational telemetry. A future tenant-level Fail-Closed policy may be supported for workloads that require stricter enforcement.

NFR-5.4 Scalability

ZeroPoint shall use externally managed state and remain horizontally scalable across edge execution locations. No request shall depend on mutable state held by a particular Worker instance.

NFR-5.5 Observability

```
requests_total
requests_allowed
requests_rejected
budget_rejections
circuit_breaker_events
redis_failures
queue_failures
provider_errors
streaming_requests
token_usage
estimated_spend
```

NFR-5.6 Maintainability

Provider integrations shall be modularized behind a common provider-adapter interface.

```
interface LLMProviderAdapter {
    authenticate(request: Request): Promise<boolean>;
    buildProviderRequest(request: Request): Promise<Request>;
    parseStreamChunk(chunk: Uint8Array): ProviderUsage | null;
    calculateCost(usage: ProviderUsage): number;
}
```

6. API Behavior

6.1 Successful Request

```
POST /v1/chat/completions
Authorization: Bearer pk_live_xxxxx
x-user-id: user_123
Content-Type: application/json

HTTP/1.1 200 OK
Content-Type: text/event-stream
```

6.2 Authentication Failure

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json

{
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Invalid or missing credentials."
  }
}
```

6.3 Budget Failure

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json

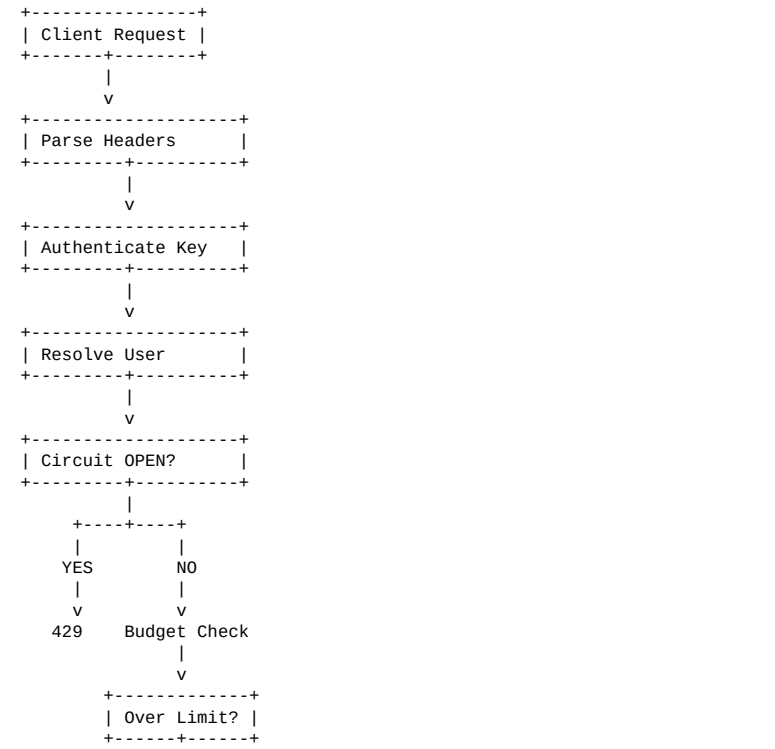
{
  "error": {
    "code": "BUDGET_EXCEEDED",
    "message": "User spending limit exceeded."
  }
}
```

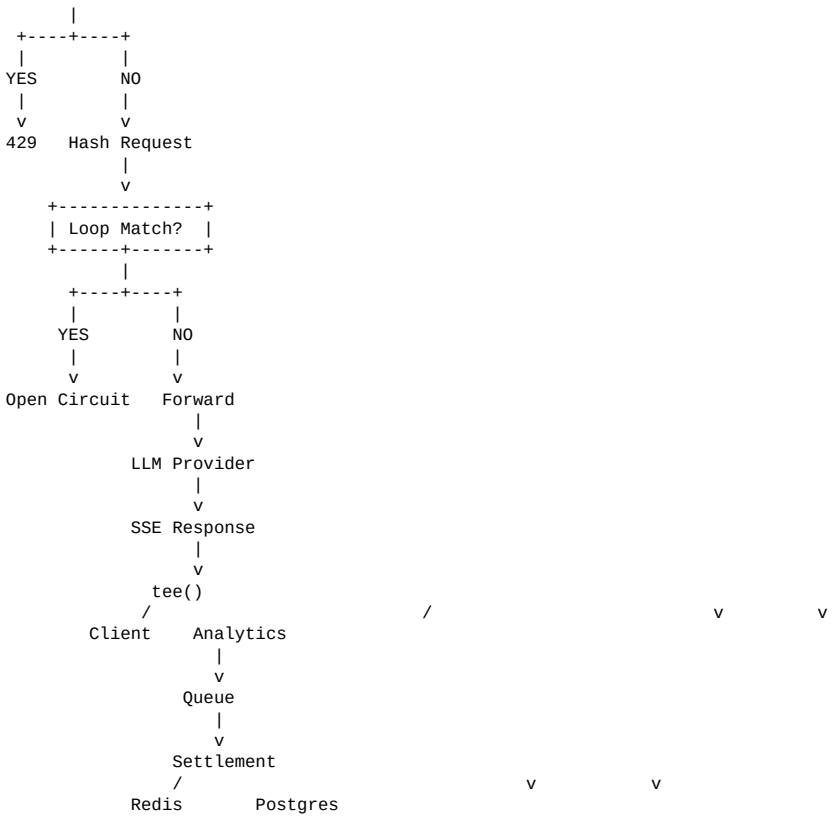
6.4 Circuit Breaker Failure

```
HTTP/1.1 429 Too Many Requests
Retry-After: 900

{
  "error": {
    "code": "CIRCUIT_OPEN",
    "message": "AI request temporarily blocked by ZeroPoint protection."
  }
}
```

7. End-to-End Request Lifecycle





8. Error Handling Requirements

Category	Examples	Required Behavior
Client Errors	400, 401, 403, 429	Return sanitized client-safe response
Provider Errors	401, 404, 429, 500, 503	Return sanitized provider error; never expose credentials
Redis Failure	Timeout/unavailable	Apply configured fail-open strategy and emit telemetry
Queue Failure	Transient delivery failure	Retry asynchronously without affecting client stream
Duplicate Settlement	Repeated request event	Use request_id idempotency to prevent double accounting

9. Cost Calculation

ZeroPoint shall normalize provider-specific usage into a common internal representation and calculate costs from model pricing.

```
input_cost  = input_tokens  × input_price_per_token
output_cost = output_tokens × output_price_per_token
total_cost  = input_cost + output_cost
```

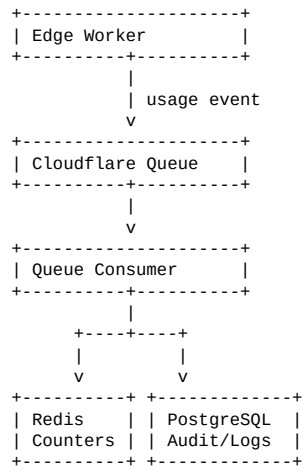
```
interface ModelPricing {
    provider: string;
    model: string;
    inputPerMillionTokens: number;
    outputPerMillionTokens: number;
}
```

10. Redis Data Model

Key Pattern	Purpose
spend:{tenant_id}:{user_id}	Current spend counter

Key Pattern	Purpose
circuit:{tenant_id}:{user_id}	Circuit state and expiration
velocity:{tenant_id}:{user_id}	60-second spending window
hashes:{tenant_id}:{user_id}	Last five request hashes
settled:{request_id}	Settlement idempotency

11. Queue & Settlement Architecture



The queue decouples user-facing latency from analytics persistence. Queue retries shall be safe through idempotent request identifiers.

12. Administrative Functions

• Create tenants and configure governance policy.

• Generate and revoke proxy keys.

• Configure provider credentials.

• Create/configure end-user budgets.

• View aggregate and user-level spending.

• View request volume and token consumption.

• View circuit-breaker events.

• Configure webhook destinations.

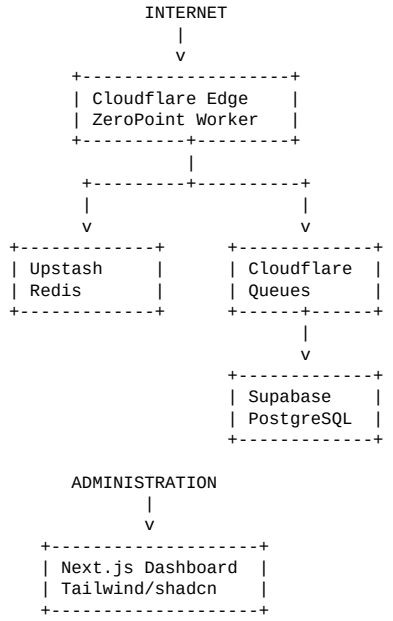
• Review operational failures.

13. Functional Requirements Traceability

Requirement	Phase	Component	Priority
FR-1.1 Header Authentication	1	Edge Worker	Critical
FR-1.2 Budget Check	1	Edge Worker + Redis	Critical
FR-1.3 Header Transformation	1	Edge Worker	Critical
FR-1.4 Provider Routing	1	Edge Worker	High
FR-2.1 Dual-Stream Piping	2	Edge Worker	Critical
FR-2.2 SSE Decoder	2	Edge Worker	High
FR-2.3 Queue Settlement	2	Queue + Redis	Critical

Requirement	Phase	Component	Priority
FR-2.4 Idempotent Settlement	2	Queue + Redis	High
FR-3.1 Burn Rate	3	Redis	Critical
FR-3.2 Velocity Hard Stop	3	Edge Worker	Critical
FR-3.3 Pattern Hashing	3	Edge Worker + Redis	Critical
FR-3.4 Circuit State Machine	3	Edge Worker + Redis	Critical
FR-3.7 Webhooks	3	Queue/Worker	High
FR-4.1 Key Provisioning	4	Dashboard + PostgreSQL	Critical
FR-4.2 Analytics	4	Next.js + PostgreSQL	High

14. Deployment Architecture



15. Recommended Logical Project Structure

```
zeropoint/
|
+-- apps/
|   +-- worker/src/
|   |   +-- index.ts
|   |   +-- router.ts
|   |   +-- auth.ts
|   |   +-- budget.ts
|   |   +-- circuit-breaker.ts
|   |   +-- proxy.ts
|   |   +-- streaming/
|   |       +-- sse-parser.ts
|   |       +-- usage-extractor.ts
|   |
|   +-- dashboard/
|       +-- app/
|       +-- components/
|       +-- lib/
|
+-- packages/
|   +-- providers/
|   +-- pricing/
|   +-- schemas/
|   +-- crypto/
|   +-- types/
|
+-- database/migrations/
+-- infrastructure/
+-- docs/
+-- tests/
```

16. Testing Requirements

16.1 Unit Testing

• Header parsing

• Proxy-key hashing

• Authentication

• Budget comparison

• Cost calculation

• SHA-256 request hashing

• Circuit transitions

• SSE parsing

• Provider adapters

16.2 Integration Testing

Integration tests shall exercise Client -> Worker -> Redis/Queue -> Mock Provider paths, including valid/invalid authentication, budget failures, Redis failure, provider failure, streaming, duplicate queue delivery, velocity breaches, hash-loop detection, and circuit expiration.

16.3 Load Testing

Load tests shall evaluate concurrent requests, concurrent SSE streams, Redis contention, queue throughput, large request bodies, high-frequency users, many tenants, and circuit-breaker state contention.

17. Acceptance Criteria

• Phase 1: valid keys authenticate; invalid/missing credentials return 401; over-budget requests return 429 before provider invocation; provider credentials replace proxy credentials.

• Phase 2: streaming is not fully buffered; tee() produces an analytics branch; SSE parsing handles arbitrary chunk boundaries; queue settlement is atomic and idempotent.

• Phase 3: 60-second velocity is tracked; the default $2.00/60s threshold opens the circuit; three consecutive matching hashes trigger loop protection; requests remain blocked for 15 minutes; webhook alerts are generated.

• Phase 4: proxy keys are shown only once; only SHA-256 digests are persisted; dashboard displays spending and circuit events; prompt/output text never appears in analytics storage.

18. Architectural Risks & Mitigations

Risk	Impact	Mitigation
Redis outage	Guardrails temporarily unavailable	Fail-open + operational alerting
Queue delay	Spend state eventually consistent	Atomic settlement + bounded queue latency
Provider usage unavailable	Incorrect cost calculation	Provider adapters + fallback
Streaming parser bug	Lost usage accounting	Provider-specific parser tests
Duplicate queue delivery	Double charging	Idempotent request IDs
Credential exposure	Critical security issue	Encryption + secret isolation + redacted logs
Tenant key collision	Cross-tenant access	Tenant-scoped Redis keys
Runaway agent	Excessive API expenditure	Velocity breaker + pattern detection
False loop detection	Legitimate requests blocked	Careful normalization + configurable thresholds
Pricing changes	Incorrect cost calculations	Versioned pricing configuration

19. Future Extensions

• Multi-level budgets: tenant, user, API key, model, and feature.

• Model-aware governance and model-specific restrictions.

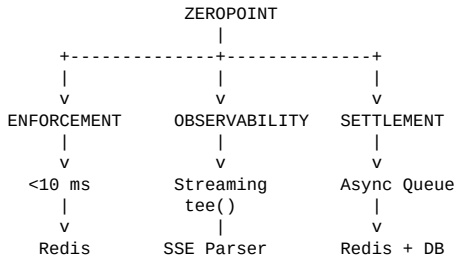
• Dynamic budgets based on subscription tier and account policy.

• Advanced agent detection using retry frequency, tool-call frequency, token velocity, and execution depth.

• HALF_OPEN circuit recovery state for controlled probes.

20. Final System Definition

ZeroPoint is an edge-native financial control plane for LLM consumption. Its architecture separates real-time enforcement from eventual analytics, allowing the system to protect SaaS economics without becoming a perceptible bottleneck for legitimate AI traffic.



Core guarantees: unauthorized requests are rejected; budget-exceeded requests are blocked before provider expenditure; provider credentials remain hidden; streaming is forwarded without full buffering; usage is settled asynchronously; runaway spending can trip a circuit breaker; repeated agent requests can be detected without storing prompt bodies; durable analytics contain metadata rather than user content; tenant state is isolated; infrastructure failures do not unnecessarily take customer applications offline.